PortuSQL

Curso introdutório de bancos de dados

---

aprendendo a manipular dados com uma linguagem em português

para ensino médio e técnico

Roteiro do curso

Parte	Conteúdo
1	Fundamentos de bancos de dados
2	Apresentando o PortuSQL e a IDE
3	Estruturar: `CRIAR` `ALTERAR` `EXCLUIR` (DDL)
4	Manipular: `INSERIR` `ATUALIZAR` `DELETAR` (DML)
5	Consultar: `SELECIONAR` `MOSTRAR` (DQL)
6	Prática guiada: banco ESCOLA completo
7	Revisão, erros comuns e exercício

> 💡 Sugestão de duração: 2 aulas de 50 min (Partes 1–5) + 1 aula (Partes 6–7).

PARTE 1

Fundamentos de bancos de dados

O que é um banco de dados?

- **Banco de dados:** coleção organizada de informações guardadas para consulta e uso posterior.
- Exemplos do dia a dia: cadastro da escola, ficha da biblioteca, contatos do celular, notas do sistema acadêmico.

Por que não usar só arquivos (Excel/texto)?

Problema	Com arquivos soltos	Com banco de dados
Dados duplicados	Fáceis de aparecer	Controlados
Vários programas ao mesmo tempo	Corrompem o arquivo	Acesso coordenado
Regras de validação	Manuais	Automáticas
Buscar informação	Lento	Rápido e poderoso

O modelo relacional

O modelo mais usado no mundo é o **relacional**, proposto por E. F. Codd (1970):

- Os dados ficam em **tabelas** (também chamadas de *relações*).
- Cada **coluna** guarda um tipo de informação (nome, idade, data...).
- Cada **linha** (registro) é um item da tabela.

```
  alunos
+----+--------+------+
| id | nome   | idade|
+----+--------+------+
|  1 | João   |   16 |
|  2 | Maria  |   17 |
|  3 | Ana    |   15 |
+----+--------+------+
```

> 🧠 Pense numa **planilha bem organizada** — mas com superpoderes.

Chaves: primária e estrangeira

Chave primária (`CHAVE PRIMARIA`) — identifica cada linha de forma **única**.

```
id  nome     id
 1  João      1   ← repetir o id 1? ERRO — já existe!
```

Chave estrangeira (`CHAVE ESTRANGEIRA`) — liga uma tabela a outra.

```
turmas              alunos
+----+---------+    +----+-------+---------+
| id | serie   |    | id | nome  | turma_id|──→ aponta para turmas.id
+----+---------+    +----+-------+---------+
|  1 | 1º Ano  |    |  1 | João  |       1 |
```

- Garante que **só existem referências válidas**: um aluno não pode apontar para uma turma que não existe.
- No PortuSQL, FK se escreve com `REFERENCIA <tabela> ( <coluna> )`.

SGBD: o programa que cuida do banco

SGBD = Sistema Gerenciador de Banco de Dados.
É o "programa do banco": guarda os dados, aplica regras e responde consultas.

Exemplos: Oracle, PostgreSQL, MySQL, SQL Server... e o **H2**.

Característica do H2	Detalhe
Tipo	Relacional, com SQL padrão
Modo	Pode rodar **embarcado** dentro de um programa Java
Arquivos	Cada banco vira um arquivo no computador
Usado no PortuSQL	Sim — é o banco alvo da tradução

> O PortuSQL não é um banco de dados: é uma **linguagem** que traduz seus comandos para o SQL do H2.

PARTE 2

Apresentando o PortuSQL e a IDE

O problema

O SQL é a linguagem dos bancos de dados — o padrão mundial. Mas ele foi criado em **inglês**:

```
SELECT name FROM students WHERE age > 18;
```

- Para quem está aprendendo, o desafio dobrado: **a lógica** + **o idioma**.
- Palavras como `WHERE`, `INSERT` e `DELETE` atrapalham a concentração no raciocínio.

> E se pudéssemos escrever a mesma coisa assim?

```
SELECIONAR nome DE alunos ONDE idade > 18;
```

O que é o PortuSQL

- Um **dialeto de SQL em português**, feito para o **ensino** de bancos de dados.
- Um **programa** = **uma sentença** que descreve uma operação no banco.
- **Não importa maiúsculas/minúsculas**: `selecionar`, `Selecionar` e `SELECIONAR` são a mesma coisa.
- Por trás, o PortuSQL **traduz seu comando para SQL de verdade** (H2) e executa — você aprende a lógica certa, que vale para qualquer SQL.

Hoje a linguagem tem 20 comandos: criar/alterar/excluir tabelas, inserir/atualizar/deletar, selecionar, mostrar estrutura e comandos de banco.

Como funciona por dentro

Todo comando passa por **4 etapas**:

```
Você digita → Lexer → Parser → Tradutor → Executor
```

Etapa	O que faz	Erro típico desta etapa
Lexer	Quebra o texto em *tokens* (palavras, números, símbolos)	Caractere inválido, aspas não fechadas
Parser	Valida a estrutura do comando	Palavra fora de ordem, comando incompleto
Tradutor	Converte para SQL do banco H2	Tipo de dado sem tradução
Executor	Roda o SQL e devolve o resultado	Violou uma regra do banco

```
Você:     SELECIONAR nome DE alunos ONDE idade > 18
Tradutor: SELECT nome FROM alunos WHERE idade > 18;
```

> As mensagens de erro indicam **linha e coluna** — seu GPS para consertar.

A IDE do PortuSQL

O projeto inclui uma interface gráfica (em Java/Swing):

- **Homepage** com cards de acesso rápido.
- **Editor de consultas** com abas, numeração de linha e realce de sintaxe.
- Botões e atalhos para executar:

Ação	Atalho
Executar consulta	`F5` ou `Ctrl+Enter`
Executar só a linha atual	`F6` (Execução única)
Parar execução	botão Parar

- Painéis de **resultados** (grade) e **mensagens** (erros/avisos).
- Ao abrir, já conecta a um banco de teste dedicado `teste_portusql` — o seu banco real nunca é aberto sem pedir.

As famílias de comandos

Família	Para quê	Comandos
DDL — estrutura	Definir e mudar tabelas	`CRIAR TABELA` `ALTERAR TABELA` `ESVAZIAR TABELA` `EXCLUIR TABELA`
DML — dados	Guardar e mexer nas linhas	`INSERIR` `ATUALIZAR` `DELETAR`
DQL — consultas	Ler e inspecionar	`SELECIONAR` `MOSTRAR TABELA` `MOSTRAR COLUNA`
Banco	Gerenciar bancos	`USAR BANCO DE DADOS` `EXCLUIR BANCO DE DADOS`

Fluxo do dia a dia: criar a tabela → inserir dados → selecionar para consultar.

Mapa dos 20 comandos

#	Comando PortuSQL	Vira no SQL
1	`CRIAR TABELA`	`CREATE TABLE`
2–10	`ALTERAR TABELA ...` (9 ações)	`ALTER TABLE ...`
11	`ESVAZIAR TABELA`	`TRUNCATE TABLE`
12	`EXCLUIR TABELA`	`DROP TABLE`
13	`INSERIR`	`INSERT INTO`
14	`ATUALIZAR`	`UPDATE`
15	`DELETAR`	`DELETE FROM`
16	`SELECIONAR`	`SELECT`
17	`MOSTRAR TABELA`	`SHOW COLUMNS FROM`
18	`MOSTRAR COLUNA`	consulta ao `INFORMATION_SCHEMA`
19–20	`USAR` `EXCLUIR BANCO DE DADOS`	*(reconhecidos, ainda sem execução)*

PARTE 3

Estruturar as tabelas — DDL

CRIAR TABELA

A tabela é como uma **planilha**: colunas (campos) e linhas (registros).

```
PortuSQL: CRIAR TABELA alunos (
              id inteiro CHAVE PRIMARIA,
              nome texto NAO NULO,
              idade inteiro
          )

SQL     : CREATE TABLE alunos (
              id int PRIMARY KEY,
              nome text NOT NULL,
              idade int
          );
```

- Cada coluna: **nome + tipo de dado** + restrições opcionais.
- `CHAVE PRIMARIA` identifica a linha de forma única (nunca se repete).
- É preciso pelo menos **uma coluna** — tabela vazia de estrutura não existe.

Anatomia da definição de coluna

```
<nome_coluna> <tipo_dado> [CHAVE PRIMARIA] [NAO NULO]
```

Parte	Papel	Exemplo
nome	como a coluna é chamada	`nome`, `data_nasc`, `turma_id`
tipo	que dado ela guarda	`texto`, `inteiro`, `data`
`CHAVE PRIMARIA`	identificador único da linha	`id`
`NAO NULO`	obrigatório preencher	`nome`

- As restrições são **opcionais** e podem aparecer **em qualquer ordem**.
- Identificadores: letras, dígitos e `_` (não podem começar com número).
- Não use palavras reservadas (como `tabela`, `onde`, `valores`) como nome.

Tipos de dados

PortuSQL	SQL (H2)	Para guardar	Exemplo
`int` / `inteiro`	`int`	número inteiro	`20` `-3`
`texto`	`text`	texto livre	`'João'` `'1º Ano'`
`decimal`	`decimal`	número com vírgula	`3.14` `8.75`
`data`	`date`	só a data	`'2026-05-20'`
`hora`	`time`	só a hora	`'07:30'`
`data_hora`	`timestamp`	data e hora juntas	`'2026-05-20 07:30'`

⚠️ Os tipos `gint`, `cint` e `mint` são reconhecidos pelo analisador, mas ainda **não têm tradução** no banco — se usar, virá erro de tradução.
Prefira `int`, `decimal` etc.

Restrições na coluna

Duas restrições podem ser escritas **junto da coluna**, no `CRIAR TABELA` (também no `ADICIONAR COLUNA`):

Restrição	SQL	Significado
`CHAVE PRIMARIA`	`PRIMARY KEY`	Identifica a linha unicamente; não repete e não é nula
`NAO NULO`	`NOT NULL`	Obrigatório: não pode ficar vazio

```
PortuSQL: CRIAR TABELA alunos (
              id inteiro CHAVE PRIMARIA,
              nome texto NAO NULO,
              email texto
          )
```

- `id`: cada aluno tem um número próprio — **nunca repetido**.
- `nome`: não dá para cadastrar aluno sem nome.
- `email`: sem restrição — pode ficar vazio (nulo).

Restrições nomeadas — ADICIONAR RESTRICAO

Para restrições mais avançadas, usa-se o `ADICIONAR RESTRICAO` —
que dá um **nome** à regra (bom para organização e para removê-la depois):

```
ALTERAR TABELA alunos ADICIONAR RESTRICAO pk_id id chave primaria
ALTERAR TABELA alunos ADICIONAR RESTRICAO un_email email unica
```

Tipo de restrição	SQL	Para quê
`chave primaria`	`PRIMARY KEY`	Identificador único da tabela
`unica`	`UNIQUE`	Nada repetido nessa coluna (ex.: e-mail)
`chave estrangeira`	`FOREIGN KEY`	Ligação com outra tabela

```
ALTERAR TABELA alunos ADICIONAR RESTRICAO fk_turma turma_id
        chave estrangeira REFERENCIA turmas ( id )
```

> "O campo `turma_id` só pode conter ids que **existem** na tabela `turmas`."

Restrição VERIFICAR (CHECK)

O `VERIFICAR` valida uma **condição** — não uma coluna específica:

```
PortuSQL: ALTERAR TABELA alunos ADICIONAR RESTRICAO ck_idade VERIFICAR idade > 0

SQL     : Alter Table alunos Add Constraint ck_idade Check (idade > 0);
```

- Se alguém tentar inserir ou atualizar uma linha que **quebra a condição**, o banco recusa a operação.
- Outros exemplos úteis:

```
... VERIFICAR idade BETWEEN 0 AND 150   (condições em geral)
... VERIFICAR nota >= 0 E nota <= 10    (usando E, do PortuSQL)
```

🛡️ Restrições são o "segurança" do banco: erram menos, cedo, do que deixar dado sujo entrar e consertar depois.

ALTERAR TABELA — visão geral

Tabela já existe e precisa mudar? `ALTERAR TABELA` tem **9 ações**:

Ação	Exemplo
`ADICIONAR COLUNA`	`ALTERAR TABELA alunos ADICIONAR COLUNA email texto`
`REMOVER COLUNA`	`ALTERAR TABELA alunos REMOVER COLUNA email`
`ALTERAR COLUNA`	`ALTERAR TABELA alunos ALTERAR COLUNA idade decimal`
`RENOMEAR COLUNA`	`ALTERAR TABELA alunos RENOMEAR COLUNA nome PARA nome_completo`
`ADICIONAR RESTRICAO`	`... ADICIONAR RESTRICAO ck_idade VERIFICAR idade > 0`
`REMOVER RESTRICAO`	`... REMOVER RESTRICAO ck_idade`
`RENOMEAR RESTRICAO`	`... RENOMEAR RESTRICAO fk_turma PARA fk_turma_aluno`
`RENOMEAR PARA`	`ALTERAR TABELA alunos RENOMEAR PARA estudantes`

ALTERAR TABELA — detalhes e cuidados

Mudar o tipo de uma coluna:

```
PortuSQL: ALTERAR TABELA alunos ALTERAR COLUNA idade decimal
SQL     : Alter Table alunos Alter Column idade decimal;
```

Renomear coluna e tabela:

```
PortuSQL: ALTERAR TABELA alunos RENOMEAR COLUNA nome PARA nome_completo
SQL     : Alter Table alunos Rename Column nome To nome_completo;

PortuSQL: ALTERAR TABELA alunos RENOMEAR PARA estudantes
SQL     : Alter Table alunos Rename To estudantes;
```

⚠️ Atenção às diferenças:

- Renomear **tabela** é só `RENOMEAR PARA` (sem a palavra `TABELA`).
- `REMOVER COLUNA` **apaga os dados** da coluna junto — sem volta!

ESVAZIAR vs EXCLUIR

Duas formas de tirar dados — muito diferentes:

```
ESVAZIAR TABELA alunos   →  TRUNCATE TABLE alunos;   (esvazia as linhas)
EXCLUIR TABELA alunos    →  DROP TABLE alunos;       (apaga a tabela inteira)
```

	ESVAZIAR TABELA	EXCLUIR TABELA
Linhas	Apaga **todas**	Apaga **todas**
Estrutura (colunas)	**Fica**	**Some**
Depois dá para inserir?	Sim	Não — a tabela nem existe mais

🧠 Analogia: a tabela é uma **gaveta**.

- `ESVAZIAR` = esvaziar o conteúdo, a gaveta continua lá.
- `EXCLUIR` = jogar fora a gaveta com tudo dentro.

PARTE 4

Manipular os dados — DML

INSERIR — guardando a primeira linha

```
PortuSQL: INSERIR EM alunos VALORES (1, 'João', 20)

SQL     : INSERT INTO alunos VALUES (1, 'João', 20);
```

- A ordem obrigatória é `INSERIR ... EM <tabela> VALORES (...)`.
- Sem lista de colunas, os valores seguem a **ordem das colunas da tabela**.

Duas formas de escrever:

```
-- Todos os campos, na ordem da tabela:
INSERIR EM alunos VALORES (1, 'João', 20)

-- Só os campos escolhidos (os outros ficam vazios/padrão):
INSERIR (nome, idade) EM alunos VALORES ('João', 20)
```

> Erro comum: escrever `INSERIR VALORES (...)` **sem o EM** — o Parser recusa.

Valores: o que pode entrar

Tipo de valor	Como se escreve	Exemplos
Número inteiro	sem aspas	`20` `-3` `0`
Número decimal	sem aspas, com ponto	`3.14` `8.75`
Texto	**entre aspas simples**	`'João'` `'1º Ano A'`
Nulo (vazio)	a palavra `nulo`	`nulo`

```
INSERIR (nome, nota, turma_id) EM alunos VALORES ('Maria', 8.75, 1)
INSERIR (nome, nota) EM alunos VALORES ('Ana', nulo)
```

⚠️ Texto sem aspas (`VALORES (João, 20)`) é **erro léxico** —
o Lexer não sabe que `João` é um valor.
⚠️ Números com aspas (`'20'`) viram texto — quase sempre não é o que queremos.

ATUALIZAR — corrigindo dados

```
PortuSQL: ATUALIZAR alunos DEFINIR idade = 21
          ONDE nome PARECIDO COM 'João'

SQL     : UPDATE alunos SET idade = 21 WHERE nome LIKE 'João';
```

- `DEFINIR` lista o que mudar: `coluna = valor` (pode ter vários, com vírgula).
- `ONDE` diz **quais linhas** mudar.

```
-- Vários campos de uma vez:
ATUALIZAR alunos DEFINIR idade = 21, nome = 'João Pedro' ONDE id = 1
```

> ⚠️ **Sem `ONDE`, TODAS as linhas são atualizadas!**
>
> `ATUALIZAR alunos DEFINIR idade = 10` deixa todo mundo com 10 anos.
>
> Confira o filtro antes de executar.

DELETAR — removendo linhas

```
PortuSQL: DELETAR DE alunos ONDE idade < 15

SQL     : DELETE FROM alunos WHERE idade < 15;
```

Remove **linhas**, mantendo a estrutura da tabela.

```
-- Um registro específico (o mais comum):
DELETAR DE alunos ONDE id = 3
```

> ⚠️ Assim como no `ATUALIZAR`: **sem `ONDE`, todas as linhas são apagadas.**
> `DELETAR DE alunos` = tabela vazia (mas com as colunas ainda lá).

Diferença para `ESVAZIAR` / `EXCLUIR`:

`DELETAR` permite apagar **só o que você quer** (com filtro);

`ESVAZIAR` tira tudo de uma vez, sem condição.

Mão na massa · DML completo

Um ciclo real de manutenção do cadastro:

```
-- 1. Entrada de dados novos:
INSERIR (nome, idade, turma_id) EM alunos VALORES ('Carla', 16, 1)

-- 2. Correção de cadastro:
ATUALIZAR alunos DEFINIR idade = 17 ONDE nome PARECIDO COM 'Carla'

-- 3. Saída de aluno (apaga SÓ ele):
DELETAR DE alunos ONDE nome PARECIDO COM 'Carla'
```

🔄 Ciclo de vida do registro: **nasce no INSERIR, cresce no ATUALIZAR, morre no DELETAR**. Os três sempre apontam para uma **tabela** e podem filtrar com **ONDE**.

PARTE 5

Consultar os dados — DQL

SELECIONAR — o comando mais usado

```
PortuSQL: SELECIONAR * DE alunos
SQL     : SELECT * FROM alunos;
```

`*` significa **todas as colunas**. Também dá para pedir só o que interessa:

```
PortuSQL: SELECIONAR nome, idade DE alunos
SQL     : SELECT nome, idade FROM alunos;
```

Sintaxe completa (tudo depois da tabela é opcional):

```
SELECIONAR <colunas | *> DE <tabela>
     [ONDE <condicao>]
     [AGRUPAR POR <colunas>]
     [ORDENAR POR <colunas>]
```

ONDE — filtros de comparação

```
PortuSQL: SELECIONAR nome, idade DE alunos ONDE idade > 18

SQL     : SELECT nome, idade FROM alunos WHERE idade > 18;
```

Operadores de comparação:

PortuSQL	Significado
`=`	igual a
`>` `<`	maior / menor que
`>=` `<=`	maior ou igual / menor ou igual
`<>`	**diferente** de

```
SELECIONAR * DE alunos ONDE idade >= 15 E idade <= 18
SELECIONAR * DE alunos ONDE turma_id <> 1
```

ONDE — combinando condições

PortuSQL	SQL	Significado
`E`	`AND`	as duas precisam ser verdadeiras
`OU`	`OR`	pelo menos uma é verdadeira
`NAO`	`NOT`	inverte a condição

```
PortuSQL: SELECIONAR * DE alunos
          ONDE idade >= 15 E turma_id = 1

PortuSQL: SELECIONAR * DE alunos
          ONDE cidade PARECIDO COM 'Campinas' OU cidade PARECIDO COM 'Sumaré'
```

- Também existe a **concatenação** de texto com `||`:

```
SELECIONAR nome || ' - ' || cidade DE alunos
```

> 🧩 Dica de leitura: traduza `E` como "e ao mesmo tempo", `OU` como "ou então".

ONDE — procurando texto: PARECIDO COM

PortuSQL	SQL	Significado
`PARECIDO COM`	`LIKE`	"combina com o padrão"

```
PortuSQL: SELECIONAR nome DE alunos ONDE nome PARECIDO COM 'Jo%'

SQL     : SELECT nome FROM alunos WHERE nome LIKE 'Jo%';
```

Curingas do padrão:

Símbolo	Combina com
`%`	qualquer quantidade de caracteres (`'Jo%'` → João, Joana, Jonathan)
`_`	exatamente um caractere (`'Ana_'` → Ani, Ann...)

```
-- "Começa com A e termina com a":
SELECIONAR nome DE alunos ONDE nome PARECIDO COM 'A%a'
```

ORDENAR POR — a ordem dos resultados

```
PortuSQL: SELECIONAR nome, idade DE alunos ORDENAR POR nome

SQL     : SELECT nome, idade FROM alunos ORDER BY nome;
```

- Por padrão, ordem **crescente** (A→Z, 0→9).
- `AGRUPAR POR` vem **antes** de `ORDENAR POR` na sentença.

```
-- Listar alunos da maior para a menor idade:
SELECIONAR nome, idade DE alunos ORDENAR POR idade
```

📌 Regra de ordem do `SELECIONAR`:
`DE` → `ONDE` → `AGRUPAR POR` → `ORDENAR POR`.
Só `DE` é obrigatório depois das colunas.

AGRUPAR POR — resumindo resultados

```
PortuSQL: SELECIONAR cidade DE alunos AGRUPAR POR cidade

SQL     : SELECT cidade FROM alunos GROUP BY cidade;
```

- **Agrupa linhas parecidas** numa só (útil para listar valores distintos, contar por categoria etc.).

```
-- Quantos alunos por turma? (contagem por grupo):
SELECIONAR turma_id AGRUPAR POR turma_id DE alunos
```

> 💡 Na prática do dia a dia, combine com funções do banco (como contagens) para responder perguntas do tipo "quantos por categoria?".
> Hoje o foco é entender **o agrupamento em si**: linhas com o mesmo valor viram um grupo.

MOSTRAR — inspecionando a estrutura

Quando você quer ver a **estrutura** (não os dados):

```
PortuSQL: MOSTRAR TABELA alunos
SQL     : SHOW COLUMNS FROM alunos;
```

```
PortuSQL: MOSTRAR COLUNA idade DE alunos
SQL     : SELECT * FROM INFORMATION_SCHEMA.COLUMNS
          WHERE TABLE_NAME = 'ALUNOS' AND COLUMN_NAME = 'IDADE';
```

Comando	O que devolve
`MOSTRAR TABELA <t>`	Todas as colunas da tabela e seus metadados
`MOSTRAR COLUNA <c> DE <t>`	Os detalhes de uma coluna específica

🔎 Use antes de escrever consultas: confira os nomes e tipos das colunas.

Comandos de banco de dados

Comando	Situação atual
`USAR BANCO DE DADOS <nome>`	Reconhecido, mas **ainda não gera SQL** (o H2 embarcado não tem comando `USE`). A troca de banco é feita pela interface.
`EXCLUIR BANCO DE DADOS <nome>`	Reconhecido, mas o tradutor **bloqueia de propósito**. A exclusão é feita pela interface gráfica.

- No PortuSQL, cada "banco de dados" é um **arquivo H2** dentro da pasta de bancos do programa.

> 🧠 Assim você conhece os **20 comandos** — e sabe quais ainda estão em construção. Faz parte do mundo real: linguagens evoluem!

PARTE 6

Prática guiada — banco ESCOLA

Passo 1 · planejar as tabelas

Queremos guardar: turmas e alunos, com a ligação entre eles.

```
turmas                    alunos
+----+----------+         +----+----------+------+----------+
| id | serie    |         | id | nome     | idade| turma_id |
+----+----------+         +----+----------+------+----------+
|  1 | 1º Ano A |         |  1 | João     |  16  |    1     |
|  2 | 2º Ano B |         |  2 | Maria    |  17  |    1     |
+----+----------+         |  3 | Ana      |  16  |    2     |
     ↑                   +----+----------+------+----------+
     └──────────────────────────── turma_id aponta para cá
```

Decisões de projeto:

- `id` em ambas = `CHAVE PRIMARIA` (identifica unicamente).
- `nome` = `NAO NULO` (não existe aluno sem nome).
- `turma_id` = `CHAVE ESTRANGEIRA` → `turmas(id)`.

Passo 2 · criar a estrutura

```
CRIAR TABELA turmas (
    id inteiro CHAVE PRIMARIA,
    serie texto
)

CRIAR TABELA alunos (
    id inteiro CHAVE PRIMARIA,
    nome texto NAO NULO,
    idade inteiro,
    turma_id inteiro
)
```

E a ligação entre elas (restrição nomeada `fk_turma`):

```
ALTERAR TABELA alunos ADICIONAR RESTRICAO fk_turma turma_id
         chave estrangeira REFERENCIA turmas ( id )
```

> Ordem importa: **crie** `turmas` **antes** de referenciá-la em `alunos`.

Passo 3 · inserir os dados

```
INSERIR EM turmas VALORES (1, '1º Ano A')
INSERIR EM turmas VALORES (2, '2º Ano B')

INSERIR (nome, idade, turma_id) EM alunos VALORES ('João', 16, 1)
INSERIR (nome, idade, turma_id) EM alunos VALORES ('Maria', 17, 1)
INSERIR (nome, idade, turma_id) EM alunos VALORES ('Ana', 16, 2)
```

- Note a **segunda forma** do `INSERIR`: listando as colunas, dispensa o `id` (o banco gera o valor).
- A restrição `fk_turma` garante: só entra turma que **existe**.

> 🧪 Experimente errar de propósito:
> `INSERIR (nome, idade, turma_id) EM alunos VALORES ('Léo', 15, 99)`
> → o banco recusa: não existe turma 99!

Passo 4 · consultar

```
-- Todos os alunos, todas as colunas:
SELECIONAR * DE alunos

-- Só o que interessa:
SELECIONAR nome, idade DE alunos

-- Filtrando:
SELECIONAR nome DE alunos ONDE turma_id = 1

-- Ordem alfabética:
SELECIONAR nome DE alunos ONDE idade >= 16 ORDENAR POR nome
```

```
SQL equivalente da última:
SELECT nome FROM alunos WHERE idade >= 16 ORDER BY nome;
```

🔎 Depois de consultar, use `MOSTRAR TABELA alunos` para revisar a estrutura quando esquecer um nome de coluna.

Passo 5 · manter os dados

```
-- Promoção de turma (vários alunos de uma vez):
ATUALIZAR alunos DEFINIR turma_id = 2 ONDE turma_id = 1

-- Correção de cadastro:
ATUALIZAR alunos DEFINIR idade = 17 ONDE nome PARECIDO COM 'Ana'

-- Saída da escola (apaga SÓ o aluno):
DELETAR DE alunos ONDE nome PARECIDO COM 'João'
```

```
-- Mudança de estrutura no meio do caminho:
ALTERAR TABELA alunos ADICIONAR COLUNA email texto
ALTERAR TABELA alunos ADICIONAR RESTRICAO ck_idade VERIFICAR idade > 0
```

⚠️ Repare no cuidado: cada `ATUALIZAR` / `DELETAR` tem `ONDE` — sem ele, a mudança atinge **a tabela inteira**.

PARTE 7

Revisão, erros comuns e exercício

Palavras reservadas

Estas palavras **não podem** ser nomes de tabela, coluna ou restrição:

```
adicionar    agrupar      alterar      apagar       banco
chave        coluna       criar        dados        de
definir      deletar      em           e            esvaziar
estrangeira  excluir      inserir      mostrar      nao
nulo         onde         ordenar      ou           para
por          primaria     referencia   remover      renomear
restricao    selecionar   tabela       unica        usar
valores      verificar
```

- A linguagem **não diferencia maiúsculas** — `onde`, `ONDE`, `Onde` são a mesma palavra.
- A linguagem **não tem comentários** no estágio atual.
- Uma sentença por comando: escreva um comando por vez (enter não continua o comando).

Erros comuns (e como evitar)

⚠️ Armadilha	O que acontece	Como evitar
Texto sem aspas: `VALORES (João, 20)`	Erro léxico	Use `'João'`
`INSERIR VALORES (...)` sem o `EM`	Erro de sintaxe	Ordem: `EM tabela VALORES`
Tipos `gint`, `cint`, `mint`	Erro de tradução	Use `int`, `texto`, `decimal`...
`ATUALIZAR` `DELETAR` sem `ONDE`	Afeta **todas** as linhas	Sempre confira o filtro
`RENOMEAR TABELA ...` para tabela	Erro de sintaxe	Renomear tabela é `RENOMEAR PARA`
Tentar `JOIN` `HAVING` subconsultas	Não existem hoje	Veja "próximos passos"

Regras de ouro

1. **Uma sentença por comando** — e maiúsculas/minúsculas não importam.
2. **Texto entre aspas simples** `'...'`; números, sem aspas.
3. `INSERIR` sempre com `EM <tabela> VALORES` na ordem.
4. `ATUALIZAR` e `DELETAR` **quase sempre** levam `ONDE`.
5. Estrutura muda com **DDL** (`CRIAR` / `ALTERAR` / `EXCLUIR`);
   dados mudam com **DML** (`INSERIR` / `ATUALIZAR` / `DELETAR`);
   leitura é **DQL** (`SELECIONAR` / `MOSTRAR`).
6. Leia o erro com calma: ele aponta **linha e coluna**.

Resumo · tradução rápida

PortuSQL	SQL
`CRIAR TABELA ...`	`CREATE TABLE ...`
`ALTERAR TABELA ... ADICIONAR COLUNA ...`	`ALTER TABLE ... ADD COLUMN ...`
`ALTERAR TABELA ... ADICIONAR RESTRICAO ...`	`ALTER TABLE ... ADD CONSTRAINT ...`
`ALTERAR TABELA ... REMOVER COLUNA ...`	`ALTER TABLE ... DROP COLUMN ...`
`ESVAZIAR TABELA t`	`TRUNCATE TABLE t`
`EXCLUIR TABELA t`	`DROP TABLE t`
`INSERIR ... EM t VALORES (...)`	`INSERT INTO t ... VALUES (...)`
`ATUALIZAR t DEFINIR ...`	`UPDATE t SET ...`
`DELETAR DE t [ONDE ...]`	`DELETE FROM t [WHERE ...]`
`SELECIONAR ... DE t [ONDE ...]`	`SELECT ... FROM t [WHERE ...]`
`E` / `OU` / `NAO` / `PARECIDO COM`	`AND` / `OR` / `NOT` / `LIKE`

Exercício proposto

Crie o banco de uma **biblioteca**:

1. Crie a tabela `livros` com: `id` (inteiro, chave primária), `titulo` (texto, não nulo), `autor` (texto), `ano` (inteiro) e `disponivel` (texto).
2. Insira 5 livros reais (misture anos e autores).
3. Selecione apenas os títulos dos livros com `ano > 2000`.
4. Ordene a lista de livros por `titulo`.
5. Atualize o campo `disponivel` de um livro para `'nao'`.
6. Apague um livro específico pelo `id`.
7. **Desafio:** adicione a restrição `VERIFICAR ano > 1400` e tente inserir um livro do ano 1000 — o que acontece?

Próximos passos

O PortuSQL continua evoluindo. Alguns pontos já planejados:

- `USAR BANCO DE DADOS` e `EXCLUIR BANCO DE DADOS` executáveis pela linguagem.
- Mais tipos de dado (`booleano`, `enum`, `decimal` com precisão...).
- Funções de agregação (contar, somar, média) junto do `AGRUPAR POR`.
- Talvez: junções entre tabelas (`JUNTAR COM`?).

Para estudar mais: a linguagem tem documentação completa no projeto —

- `COMANDOS.md` — referência de todos os comandos, com exemplos.
- `ESPECIFICACAO.md` — especificação formal da linguagem (gramática e semântica).

> O SQL que você aprendeu aqui é o mesmo que roda o mundo —
> continue praticando!

Bora praticar!

Agora é com você: a lógica é a mesma do SQL de verdade — só que na sua língua.